Part 1: Theoretical Questions

May 25, 2024

1. (a) i. **Imperative programming:** Focuses on describing step-by-step instructions / commands the computer should execute to reach a specific goal.
      ii. **Procedural programming:** Organizes code into small, easily testable procedures or functions, which can be called from different parts of the code to promote reusability and modularity.
      iii. **Functional programming:** Emphasizes the use of pure functions and immutable data to promote declarative programming. the program is an expression (or a series of expressions) and not a series of commands. Executing the program will be finding expression's value rather then preforming actions. to avoid side effects, functions are also expressions - making everything immutable.
   (b) As apposed to imperative paradigm, the procedural paradigm adds structure and enhances code organization by breaking it to more smaller, manageable blocks of code, which allows for easier testing.
   (c) As apposed to procedural paradigm, the imperative paradigm promoting immutability and pure functions, higher-order functions and concurrency. leading to more scalable, predictable and maintainable code then in procedural programming.

2.
```
const getDiscountedProductAveragePrice = (inventory: Product[]): number => {
    const discountedPrices = inventory.filter((product) =>
        product.discounted).map((product) => product.price);
    // Filter discounted products and extract prices of discounted products

    // Ceck for empty array
    return discountedPrices.length === 0 ? 0 :
    // Calculate average
    discountedPrices.reduce((sum, price) => sum+price, 0) / discountedPrices.length;
};
```

3. (a) `(x: T[], y: (value: T) => boolean) => boolean`
   (b) `(x: number[]) => number`
   (c) `<T>(x: boolean, y: [T, T]) => T`
   (d) `<T, U>(f: (arg: U) => T, g: (arg: number) => U) => (x: number) => T`